
Software Requirements Specification

for

Insta Buddy Mart

Version 3.0 Approved by
Insta Buddy Mart Alpha Manager Team
comprising of **Tahseen Muneer Parray** and **Prajwal Pathade**

Master Spreadsheet: [Sheet Link](#)
Source Code Repository: [Github Link](#)

Prepared by

Shams Quamar, Prajwal Pathade,
Tahseen Muneer Parray, Mohit Gupta

Insta Buddy Mart Organization

Feb 27, 2026

Content

1	Introduction	3
1.1	Purpose	3
1.2	Document Conventions	3
1.3	Intended Audience and Reading Suggestions	3
1.4	Product Scope	3
1.5	References	3
2	Overall Description	3
2.1	Product Perspective	3
2.2	Product Functions	4
2.3	User Classes and Characteristics	4
2.4	Operating Environment	4
2.5	Design and Implementation Constraints	4
2.6	User Documentation	4
2.7	Assumptions and Dependencies	4
2.8	Design Standards and Conventions	4
3	External Interface Requirements	5
3.1	User Interfaces	5
3.2	Hardware Interfaces	5
3.3	Software Interfaces	5
3.4	Communications Interfaces	5
4	System Features	5
4.1	Feature: Request Creation (Requester)	5
4.1.1	Description and Priority	5
4.1.2	Stimulus/Response Sequences	5
4.1.3	Functional Requirements	6
4.2	Feature: Request Acceptance (Runner)	6
4.2.1	Description and Priority	6
4.2.2	Stimulus/Response Sequences	6
4.2.3	Functional Requirements	6
4.3	Feature: Bill Verification & Payment	6
4.3.1	Description and Priority	6
4.3.2	Stimulus/Response Sequences	6
4.3.3	Functional Requirements	7
5	Other Nonfunctional Requirements	7
5.1	Performance Requirements	7
5.2	Safety Requirements	7
5.3	Security Requirements	7
5.4	Software Quality Attributes	7
5.5	Business Rules	7

6 Other Requirements	7
6.1 Project Management	7
7 Sprint	7
A Glossary	12
B Analysis Models	13
C Requirements Traceability Matrix	18

Revision History

Name	Date	Reason For Changes	Version
Mohit Gupta, Prajwal Pathade, Shams Quamar, Tahseen Muneer Parray	21-01-2026	Initial release of the Software Requirements Specification (SRS) for Insta Buddy Mart.	1.0
Tahseen Muneer Parray, Prajwal Pathade, Mohit Gupta, Shams Quamar	04-02-2026	Added REQ-8 to REQ-16. Changed the analysis model diagrams to maintain consistency with the changes.	2.0
Tahseen Muneer Parray, Prajwal Pathade, Mohit Gupta, Shams Quamar	25-02-2026	Initial architecture established initial microservices and added Tech Stack for backend, frontend and other services and also updated the Sprint Plan to keep it in sync with the architecture.	3.0

1. Introduction

1.1 Purpose

The purpose of this document is to specify the software requirements for Insta Buddy Mart, a hyperlocal peer-to-peer delivery application. This document covers the functionality for Version 1.0 (MVP), detailing the interaction between the "Requester" and "Runner"(Deliverer) user roles.

1.2 Document Conventions

This document follows the **IEEE Std 830-1998** recommended practice for Software Requirements Specifications, utilizing the Karl E. Wiegers template structure.

Typography: Standard bolding is used for headings.

Lists: Bullet points denote non-sequential items.

Priorities: Requirements are prioritized as High, Medium, or Low (see Section 4 for detailed definitions).

Requirements are indexed with the tag **REQ-x** (Requirement) or **TBD-x** (To Be Determined). Input validation constraints are noted within the functional requirement descriptions.

1.3 Intended Audience and Reading Suggestions

This document is intended for project managers, UI/UX designers, and backend/frontend developers.

Developers: Focus on Section 3 (Interfaces) and Section 4 (System Features).

Project Managers: Focus on Section 2 (Overall Description) and Section 5 (Non-functional Requirements).

1.4 Product Scope

Insta Buddy Mart is a community-driven marketplace that connects people who need items (Requesters) with neighbors already visiting stores (Runners). The objective is to facilitate quick, low-cost deliveries while providing earning opportunities for community members. The software allows for request broadcasting, real-time matching, in-app communication, and secure escrow payments.

1.5 References

Insta Buddy Mart Wireframe Sketches (TBD)

Insta Buddy Mart Use Case Documentation (TBD).

2. Overall Description

2.1 Product Perspective

This product is a new, self-contained mobile application for Android. It relies on external interfaces for Payments (Stripe/Razor pay).

2.2 Product Functions

The major functions of the system include:

User Authentication & Escrow Model: Verification of identity and fraud-proof payments.

Request Creation: Posting shopping lists and setting delivery fees.

Order Fulfillment: Acceptance, bill uploading, and delivery confirmation.

Financial Transaction: Escrow-based payment holding and release.

2.3 User Classes and Characteristics

The Requester: Individuals unable to leave their location (due to health, time, etc.) who engage the app to buy goods.

The Deliverer: Individuals already visiting markets who wish to earn extra money.

The Administrator: Internal staff managing disputes and user verification.

2.4 Operating Environment

Client: Android (v10+) Mobile Devices.

Server: Cloud-based backend (e.g., AWS/Firebase).

Connectivity: Requires active 4G/5G or Wi-Fi connection.

2.5 Design and Implementation Constraints

Regulatory: Must comply with local financial regulations regarding P2P payments.

Privacy: Phone numbers must be masked; all chat must be internal.

2.6 User Documentation

In-app onboarding tutorial (walkthrough).

FAQ section within the Settings menu.

2.7 Assumptions and Dependencies

Assumption: Users will truthfully report the cost of items via bill images.

Dependency: The system depends on the uptime of third-party Payment Gateways.

2.8 Design Standards and Conventions

2.8.1 Naming Conventions : To ensure code consistency across the *Insta Buddy Mart* Android application and backend, the following naming conventions will be used:

Element	Naming Style	Standard	Project Examples
Classes	PascalCase	Nouns, singular	Request, Payment, Bill, User
Methods	CamelCase	Verb-Noun pair	createRequest(), verifyBill(), holdFunds()
Variables	CamelCase	Descriptive nouns	walletBalance, deliveryFee, escrowStatus
Constants	UpperCase	Fixed values	MAX_DELIVERY_DISTANCE, COMMISSION_RATE
Database	LowerCase	Plural nouns	users, transaction_logs

2.8.2 Commenting & Documentation Patterns

a) **Method Headers:** Every critical function must follow this Javadoc/KDoc style:

```
java
--+
/**
 * Verifies the bill image uploaded by the Runner.
 * @param billId - The unique ID of the bill being verified.
 * @param finalAmount - The amount entered by the Runner.
 * @return Boolean - Returns true if the Requester approves the amount.
 */
public Boolean verifyBill(String billId, Double finalAmount) { ... }
```

b) **Inline Comments:** Use `//` for logic explanations.

2.8.3 Code Formatting:

a) **Indentation:** 4 spaces.

2.8.4 Error Handling:

All external calls (e.g., Stripe/Razorpay, Firebase) must be wrapped in **try-catch** blocks.

2.8.5 Logs:

Use a standardized tag for efficient log filtering and debugging: `Log.d("InstaBuddy_Payment", "Escrow locked for RequestID: " + requestId);`

3. External Interface Requirements

3.1 User Interfaces

Mobile App: The interface will rely on touch inputs and a List Entry view for Requesters.

Standard Elements: Bottom navigation bar, Floating Action Button (FAB) for "New Request," and standard chat interfaces.

3.2 Hardware Interfaces

Camera: Required for scanning institute ID and photographing receipts/bills.

3.3 Software Interfaces

Payment Gateway: To handle credit/debit/UPI transactions.

Push Notification Service: (e.g., FCM) for real-time alerts.

3.4 Communications Interfaces

Protocol: HTTP/HTTPS for REST API calls.

Real-time: WebSocket or MQTT for Chat and Location updates.

Security: All data in transit must be encrypted via TLS 1.2+.

4. System Features

4.1 Feature: Request Creation (Requester)

4.1.1 Description and Priority

Priority: High. Allows the Requester to broadcast their needs to the community.

4.1.2 Stimulus/Response Sequences

Stimulus: User clicks " New Request."

Response: System presents a form for Store Name, Item List, and Estimated Cost.

Stimulus: User submits request.

Response: System validates data and broadcasts to nearby Deliverers.

4.1.3 Functional Requirements

- **REQ-1:** The system shall allow users to enter free-text items or upload an image of a handwritten list.
- **REQ-2:**

Delivery Fee Calculation: The system shall automatically suggest a delivery fee

based on the distance between the Store and the Requester.

Validation: The user may adjust this fee, but the system shall enforce a hard *Minimum Floor* (e.g., ₹ 30) below which the fee cannot be set.

- **REQ-3:** The system shall calculate the "Total Authorization Amount" as: (Estimated Item Cost * 1.15) + Delivery Fee. The extra 15% serves as a buffer for price variance.

4.2 Feature: Request Acceptance (Runner)

4.2.1 Description and Priority

Priority: High. Allows a Runner to view and accept jobs.

4.2.2 Stimulus/Response Sequences

Stimulus: Runner views "Nearby Requests" feed.

Response: System displays requests sorted by distance.

Stimulus: Runner clicks "Accept."

Response: System locks the request (or puts it in "Pending Approval" state).

4.2.3 Functional Requirements

- **REQ-4:** The system shall only show requests within a radius configurable by the Runner within a range of 1 km to 10 km
- **REQ-5:** The system shall prevent a Runner from accepting a request if their KYC is incomplete.

4.3 Feature: Bill Verification & Payment

4.3.1 Description and Priority

Priority: Critical. Ensures the correct amount is paid based on the actual store receipt.

4.3.2 Stimulus/Response Sequences

Stimulus: Runner uploads photo of bill and enters final amount.

Response: Requester receives notification to review the bill.

Stimulus: Requester approves amount.

Response: System processes the payment.

4.3.3 Functional Requirements

- **REQ-6.1:** If the Final Item Cost entered by the Runner is less than or equal to the Total Authorization Amount, the system shall capture the exact final amount and release the remaining buffer back to the Requester immediately.

- **REQ-6.2:** If the Final Item Cost exceeds the Total Authorization Amount, the system shall prevent the Runner from submitting the bill. The Runner must request a "Top-Up Approval" from the Requester via chat, requiring the Requester to authorize a secondary transaction for the difference.
- **REQ-7:** The system must secure the funds in an escrow state until delivery is confirmed.

4.4 Feature: Escrow & Payment Processing

4.4.1 Description and Priority

Priority: Critical Secures funds before the Runner shops to guarantee payment, holding them in a neutral "Escrow" state until delivery is confirmed.

4.4.2 Stimulus/Response Sequences

Stimulus: User confirms order.

Response: System charges User and moves funds to "Escrow Ledger."

Stimulus: User approves delivery.

Response: System moves funds from Escrow to Runner's Wallet.

Stimulus: User clicks "Reject Bill" or "Dispute."

Response: System **freezes Escrow funds** (preventing payout), flags order as "In Dispute," and redirects User to the Dispute Resolution form (Feature 4.6).

Stimulus: User takes no action for 24 hours after delivery.

Response: System **auto-releases funds** to Runner (Auto-Approval) to prevent payment holding

4.4.3 Functional Requirements

- **REQ-8:** The system shall integrate with a Payment Gateway to authorize and capture funds via UPI or Card before the Runner begins shopping.
- **REQ-9:** The system shall maintain a logical "Escrow Ledger" to segregate order funds, preventing withdrawal until delivery confirmation.
- **REQ-10:** The system shall support partial refunds to the User (if actual cost < estimated) and settlement payouts to the Runner's linked bank account.

4.5 Feature: In-Order Communication & Substitution

4.5.1 Description and Priority

Priority: Medium Enables real-time coordination for out-of-stock items, allowing Runners to suggest substitutes and Users to approve them instantly.

4.5.2 Stimulus/Response Sequences

Stimulus: Runner marks item "Unavailable" and suggests substitute.

Response: User receives notification to Approve/Reject.

Stimulus: User approves.

Response: System updates item list and cost; notifies Runner to proceed.

4.5.3 Functional Requirements

- **REQ-11:** The system shall provide a masked in-app chat or call feature connecting the Runner and Requester during the active order.
- **REQ-12:** The system shall allow the Runner to modify the item list (add/remove/substitute) exclusively while the order state is **SHOPPING**. The system must lock the order and strictly disable all list modifications the moment the Runner submits the bill photo and the state transitions to **BILL_UPLOADED**.
- **REQ-13:** The system shall prevent bill finalization if there are pending substitution requests awaiting User action.

4.6 Feature: Dispute Resolution

4.6.1 Description and Priority

Priority: Critical Handles disagreements on bill amount or delivery quality by freezing escrow funds until an Admin intervenes.

4.6.2 Stimulus/Response Sequences

Stimulus: User clicks "Report Issue" on final bill.

Response: System requests Dispute Reason and Photo Evidence.

Stimulus: User submits dispute.

Response: System freezes Escrow funds and generates a ticket for the admin.

4.6.3 Functional Requirements

- **REQ-14:** The system shall provide a "Dispute" option during the Bill Verification phase to halt the automatic release of funds.
- **REQ-15:** The system shall require Users to select a reason and upload evidence when raising a dispute.
- **REQ-16:** The system shall provide an Admin interface to review evidence and manually route escrow funds to the appropriate party (Refund or Payout).

4.7 Feature: Runner Incentive & Rating System

4.7.1 Description and Priority

Priority: High This feature rewards Runners who consistently deliver on time, maintain high ratings, and avoid disputes. The goal is to incentivize quality service, reliability, and positive user experience.

4.7.2 Stimulus/Response Sequences

Stimulus: Order is marked as Delivered and confirmed by Requester.

Response: System prompts Requester to rate Runner (1–5 stars).

Stimulus: Runner maintains high rating (≥ 4.5) over last 20 orders.

Response: System assigns "Top Runner" badge and incentive benefits.

Stimulus: Runner completes 50 successful deliveries without dispute.

Response: System grants performance bonus or commission reduction.

4.7.3 Functional Requirements

- **REQ-17:** The system shall allow Requesters to rate Runners on a 1–5 star scale after delivery confirmation.
- **REQ-18:** The system shall calculate a dynamic Runner Rating based on the weighted average of the last 20 completed orders.
- **REQ-19:** The system shall assign performance badges such as:
 - Top Runner
 - Fast Delivery
 - Trusted Runner
- **REQ-20:** The system shall provide financial incentive by reducing platform commission (3% instead of 5%)
- **REQ-21:** The system shall temporarily reduce visibility of Runners whose rating falls below 3.0 until performance improves.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

- **NFR-1 (Latency) :** Real-time updates (e.g., Order Acceptance, Chat, Status Changes) must be delivered to the end-user within $t \leq 3$ seconds under standard network conditions.
- **NFR-2 (Geographic Concurrency):** The system shall support a minimum of **1,000 concurrent active users** within a single **Logical Geographic Zone** (defined as a 5 km radius) without degradation in message delivery or request broadcast speed.
- **NFR-3 (Throughput):** The backend must be capable of processing at least **50 new request creations per minute** per zone during peak hours.

5.2 Safety Requirements

- **NFR-4 (Identity):** All "Deliverer" accounts must be verified via OTP and Institute ID/Government ID upload to reduce theft risk.

5.3 Security Requirements

- **NFR-5 (Data Masking):** User phone numbers must not be visible to the other party; an in-app bridge must be used.
- **NFR-6 (Payment Data):** No credit card information shall be stored directly on application servers (PCI-DSS compliance via Gateway).

5.4 Software Quality Attributes

- **NFR-7 (Usability):** The "Create Request" flow must be complete in fewer than 4 clicks.
- **NFR-8 (Reliability):** The app must gracefully handle network disconnects and auto-reconnect without data loss.

6. Other Requirements

6.1 Project Management

- The development team utilizes Trello as the primary tool for project management and issue tracking throughout the software development lifecycle (SDLC). Trello was selected for the following reasons:
- **Kanban Workflow:** Visualizes project status using "Backlog," "In Progress," and "Done" lists.
- **Task Assignment:** Assigns specific cards to team members to ensure accountability.
- **Sprint Planning:** Organizes User Stories into actionable tasks for weekly sprints.
- **Asset Storage:** Centralizes wireframes and notes directly within relevant task cards.

6.2 Business Rules

Commission Model: The platform operates on a "Deduction Model."

- **Requester Pays:** Item Cost + Delivery Fee.
- **Platform Takes:** 5% of Delivery Fee (deducted from the Runner's payout).
- **Runner Receives:** Item Cost (reimbursement) + (Delivery Fee * 0.95).

7. Architecture

7.1. High-Level Architecture Overview

To handle the real-time matching, secure escrow payments, and geographic concurrency requirements outlined in your SRS, the system will use a distributed microservices model.

- **Clients:** Android Native App (Requester & Runner) and Web App (Admin Dashboard).
- **API Gateway:** Acts as the single entry point for all client requests, handling rate limiting, initial token

validation, and routing.

- **Message Broker (Event Bus):** Kafka or RabbitMQ to handle asynchronous events (e.g., "Order Delivered" triggering "Release Escrow").

7.2. Core Microservices Breakdown

Each service adheres to the "Database per Service" pattern to ensure loose coupling and independent scalability.

A. Identity & Access Management (IAM) Service

- Responsibilities: User registration, OTP login, profile management, and Runner KYC validation (REQ-5).
- Tech Stack: Go or Node.js.
- Database: PostgreSQL (Relational data for user profiles).
- External Integrations: Twilio/AWS SNS (for OTP), S3 bucket (for Institute ID image storage).

B. Geolocation & Matching Service

- Responsibilities: Tracking active Runners, calculating distances, and broadcasting new requests to Runners within the 5 km radius (REQ-4, NFR-2).
- Tech Stack: Go (for high-throughput concurrency) or Python.
- Database: Redis (specifically using Redis Geospatial indexes for lightning-fast radius queries).
- Key Event: Listens for RequestCreated events to ping nearby active Runners.

C. Order Management Service

- Responsibilities: The core engine. Handles creating shopping lists (REQ-1), calculating initial delivery fees (REQ-2), managing state transitions (REQ-6), and handling item substitutions (REQ-12, REQ-13).
- Tech Stack: Java (Spring Boot) or Node.js.
- Database: PostgreSQL.
- Key Event: Emits OrderAccepted, ShoppingStarted, BillUploaded, and OrderDelivered events to the message broker.

D. Payment & Escrow Service

- Responsibilities: Highly critical service handling the Escrow Ledger (REQ-9), calculating the "Total Authorization Amount" (REQ-3), processing Auto-Refunds/Top-Ups (REQ-6.1, REQ-6.2), and distributing payouts with the 5% commission deduction.
- Tech Stack: Java (Spring Boot) - preferred for financial transactional integrity.
- Database: PostgreSQL (Strict ACID compliance required).
- External Integrations: Stripe or Razorpay APIs.
- Key Event: Listens for OrderDelivered to trigger the 24-hour Auto-Approval countdown.

E. Notification & Chat Service

- Responsibilities: Real-time masked communication between Runner and Requester (REQ-11), and system push notifications (e.g., "Substitute Suggested").
- Tech Stack: Node.js + WebSockets / Socket.io.
- Database: MongoDB (NoSQL is excellent for unstructured chat logs and high-volume messaging).
- External Integrations: Firebase Cloud Messaging (FCM) for push notifications.

F. Dispute & Admin Service

- Responsibilities: Freezing escrow funds when issues are reported, aggregating evidence (REQ-14, REQ-15), and providing endpoints for the Admin Dashboard (REQ-16).
- Tech Stack: Python (Django/FastAPI) or Node.js.
- Database: PostgreSQL.
- Key Event: Emits DisputeRaised (which tells the Payment Service to freeze the ledger) and

DisputeResolved (to trigger manual refund/payout).

7.3. Inter-Service Communication Patterns

- **Synchronous (REST/gRPC):** Used when an immediate response is required.
 - Example: The Client asks the API Gateway for the User Profile. The API Gateway makes a synchronous REST call to the IAM Service.
- **Asynchronous (Event-Driven via Kafka/RabbitMQ):** Used to decouple heavy processes and ensure system reliability.
 - Example: When a Runner uploads the final bill, the Order Service updates the order state to Bill_Uploaded and publishes an event. The Notification Service consumes this event and pushes a WebSocket alert to the Requester. The Payment Service consumes it to prepare the settlement logic.

7.4. Infrastructure & DevOps

- Containerization: Docker for packaging all microservices.
- Orchestration: Kubernetes (EKS/GKE) to manage scaling. The Geolocation and Chat services will need horizontal pod autoscaling configured to handle peak traffic hours.
- Object Storage: AWS S3 or Google Cloud Storage for storing uploaded handwritten lists, bill receipts, and KYC documents.
- Observability: Prometheus & Grafana for system metrics; ELK Stack (Elasticsearch, Logstash, Kibana) for centralized logging (crucial for debugging cross-service transactions).

8. Sprint

Sprint 1: Infrastructure, Identity, & Geolocation

Duration: February 18, 2026 – March 3, 2026 (Weeks 1 & 2)

Goal: Establish the baseline microservices infrastructure, set up the API Gateway, and deploy the foundational IAM and Geolocation services.

Task 1.1: API Gateway & IAM Service (REQ-5, NFR-4)

Assignee: Prajwal Pathade

Description: Deploy the API Gateway for request routing. Build the Identity & Access Management (IAM) Service using PostgreSQL. Implement OTP auth and an S3 bucket integration for Institute ID (KYC) uploads.

Acceptance Criteria: Client requests are successfully routed through the API Gateway. The IAM service issues valid JWTs. Runner accounts remain restricted until KYC is verified via the database.

Task 1.2: Message Broker Setup & Geolocation Service (REQ-4, NFR-2)

Assignee: Shams Quamar

Description: Spin up the Kafka/RabbitMQ event bus. Build the Geolocation Service utilizing Redis Geospatial indexes to handle high-frequency location pings.

Acceptance Criteria: Services can successfully publish and subscribe to test topics on the message broker. The Geolocation Service returns a list of active Runners within a strict 5 km radius in under 200ms.

Task 1.3: Order Management Service - Base CRUD (REQ-1, REQ-2)

Assignee: Tahseen Muneer Parray

Description: Initialize the Order Management Service (PostgreSQL) to handle the creation of shopping lists and calculate the base delivery fee.

Acceptance Criteria: Requesters can POST a new order. The service calculates the delivery fee (enforcing the ₹30 floor) and successfully emits a **RequestCreated** event to the message broker.

Sprint 2: Core Engine & Escrow Service

Duration: March 4, 2026 – March 17, 2026 (Weeks 3 & 4)

Goal: Build the highly critical financial systems, ensuring secure synchronous/asynchronous communication between the Order and Payment services.

Task 2.1: Payment & Escrow Service - Gateway Integration (REQ-3, REQ-8)

Assignee: Mohit Gupta

Description: Build the standalone Payment Service. Integrate Stripe/Razorpay SDKs to calculate and capture the "Total Authorization Amount" $((\text{Cost} * 1.15) + \text{Fee})$.

Acceptance Criteria: The service successfully authorizes card/UPI payments and creates an isolated Escrow Ledger record in its dedicated PostgreSQL database before responding to the Order Service.

Task 2.2: Event-Driven Bill Verification & State Machine (REQ-6.1, REQ-6.2, REQ-10)

Assignee: Tahseen Muneer Parray (Order Logic) & Prajwal Pathade (Payment Logic)

Description: Implement the branching logic for bill uploads. The Order Service handles the state transition, while the Payment Service listens for **BillUploaded** events to process Auto-Refunds or trigger Top-Up flows.

Acceptance Criteria: Uploading a bill lower than the authorization automatically processes a

refund delta. The system accurately deducts the 5% platform commission during payout calculations.

Task 2.3: Escrow Auto-Release Job (REQ-9)

Assignee: Mohit Gupta

Description: Configure the Payment Service to listen for the **OrderDelivered** event and initiate a 24-hour delayed background job (cron/worker) for Auto-Approval.

Acceptance Criteria: Funds strictly cannot be withdrawn while in the "Locked" state. The worker automatically releases funds to the Runner exactly 24 hours post-delivery if no **DisputeRaised** event is detected.

Sprint 3: Real-Time Communication & Dispute Microservices

Duration: March 18, 2026 – March 31, 2026 (Weeks 5 & 6)

Goal: Deploy the NoSQL communication layer, handle real-world shopping exceptions, and finalize container orchestration (Docker/K8s) for all services.

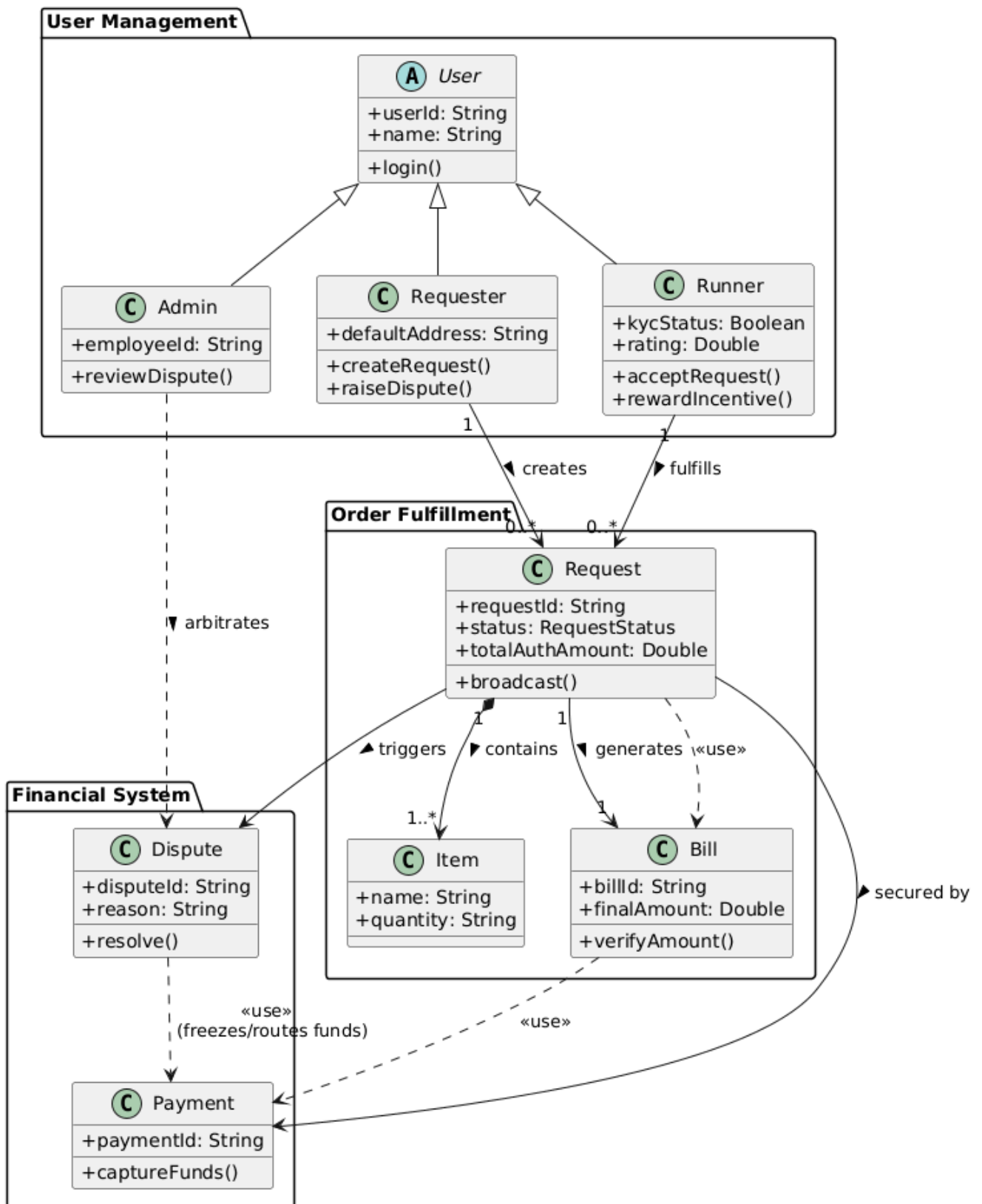
- **Task 3.1: Notification & Chat Service (REQ-11, NRF-5)**
 - **Assignee:** Shams Quamar
 - **Description:** Build the WebSocket-based Chat Service using MongoDB for unstructured message storage. Integrate Firebase Cloud Messaging (FCM) for push alerts.
 - **Acceptance Criteria:** Both parties can chat in real-time via WebSockets without exposing phone numbers. Disconnecting and reconnecting fetches the correct chat history from MongoDB using the **order_id**.
- **Task 3.2: Item Substitution & Dispute Service (REQ-12, REQ-14, REQ-16)**
 - **Assignee:** Mohit Gupta & Tahseen Muneer Pararray
 - **Description:** Update Order Service to handle substitution workflows. Build the Dispute Service to aggregate evidence and provide Admin endpoints.
 - **Acceptance Criteria:** The Dispute Service successfully emits a **DisputeRaised** event, which the Payment Service immediately consumes to change the Escrow state to "Frozen." Admins can force-release funds via the API Gateway.
- **Task 3.3: E2E Integration & Container Orchestration**
 - **Assignee:** All Team Members
 - **Description:** Finalize Dockerfiles for all 6 microservices. Set up Docker Compose for local testing and finalize environment variables for external integrations.
 - **Acceptance Criteria:** The entire system can be spun up locally using a single **docker-compose up** command, with the API Gateway successfully routing traffic to the correct containerized services.

A Glossary

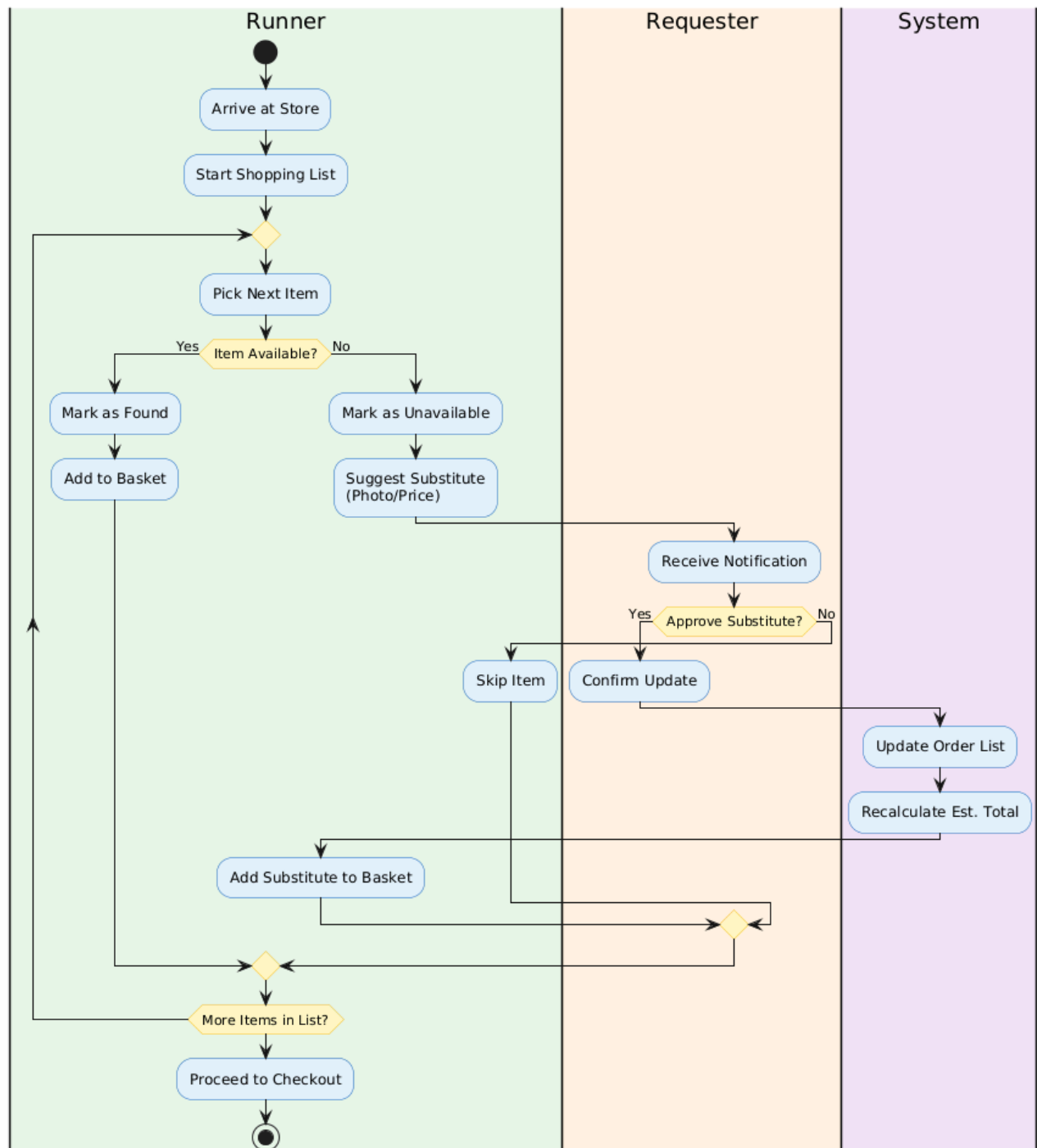
- **Requester:** User placing an order.
- **Runner:** User fulfilling an order.
- **KYC:** Know Your Customer (Identity Verification).
- **Escrow:** Financial arrangement where a third party holds money until conditions are met.

B Analysis Models

Figure 1: System Domain Class Model

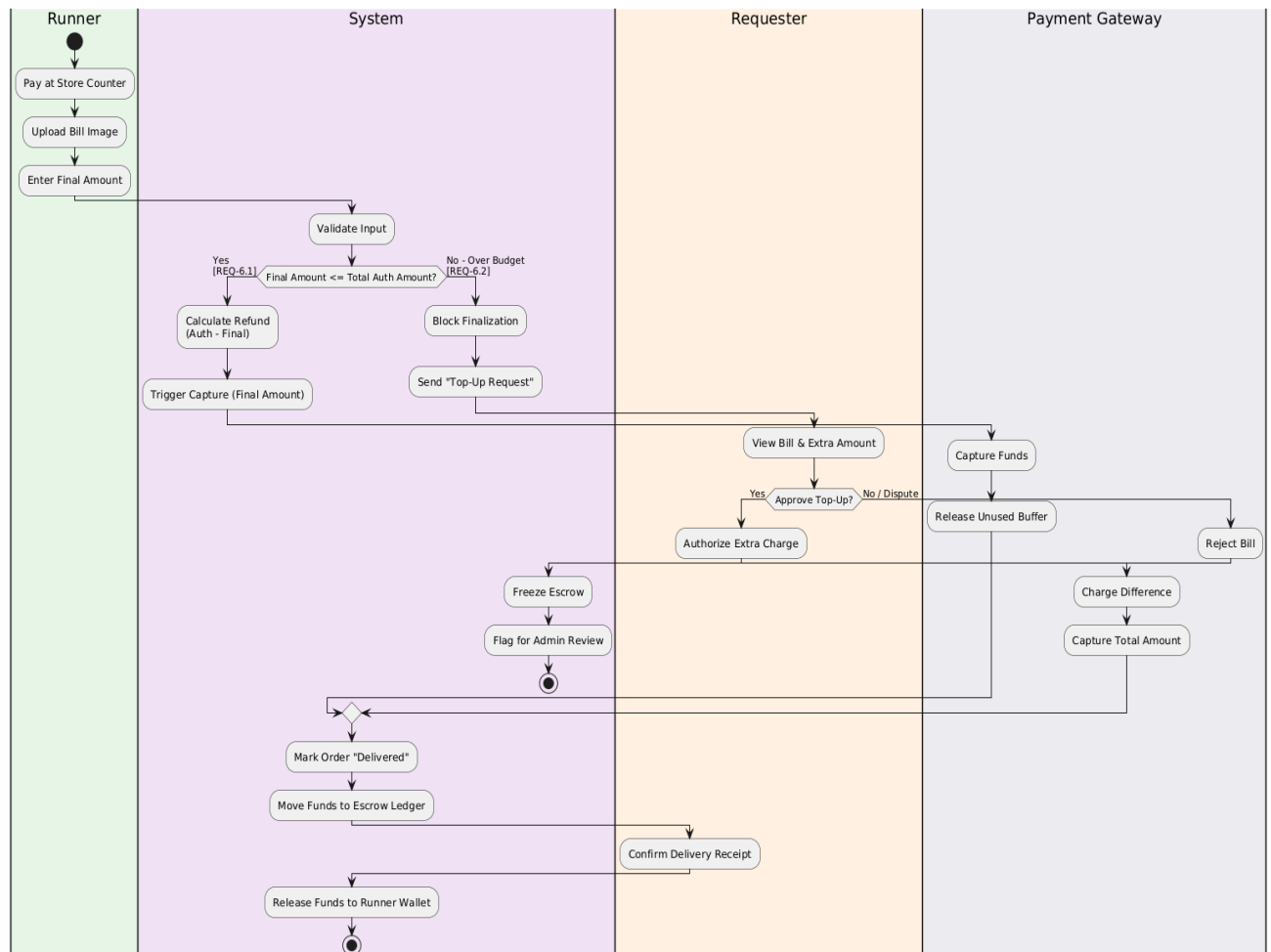


This diagram illustrates the static structure of the Insta Buddy Mart system, defining the relationships between Users (Requester, Runner, Admin), Requests, and Financial Entities (Bill, Payment, Dispute).

Figure 2: Runner Shopping & Substitution

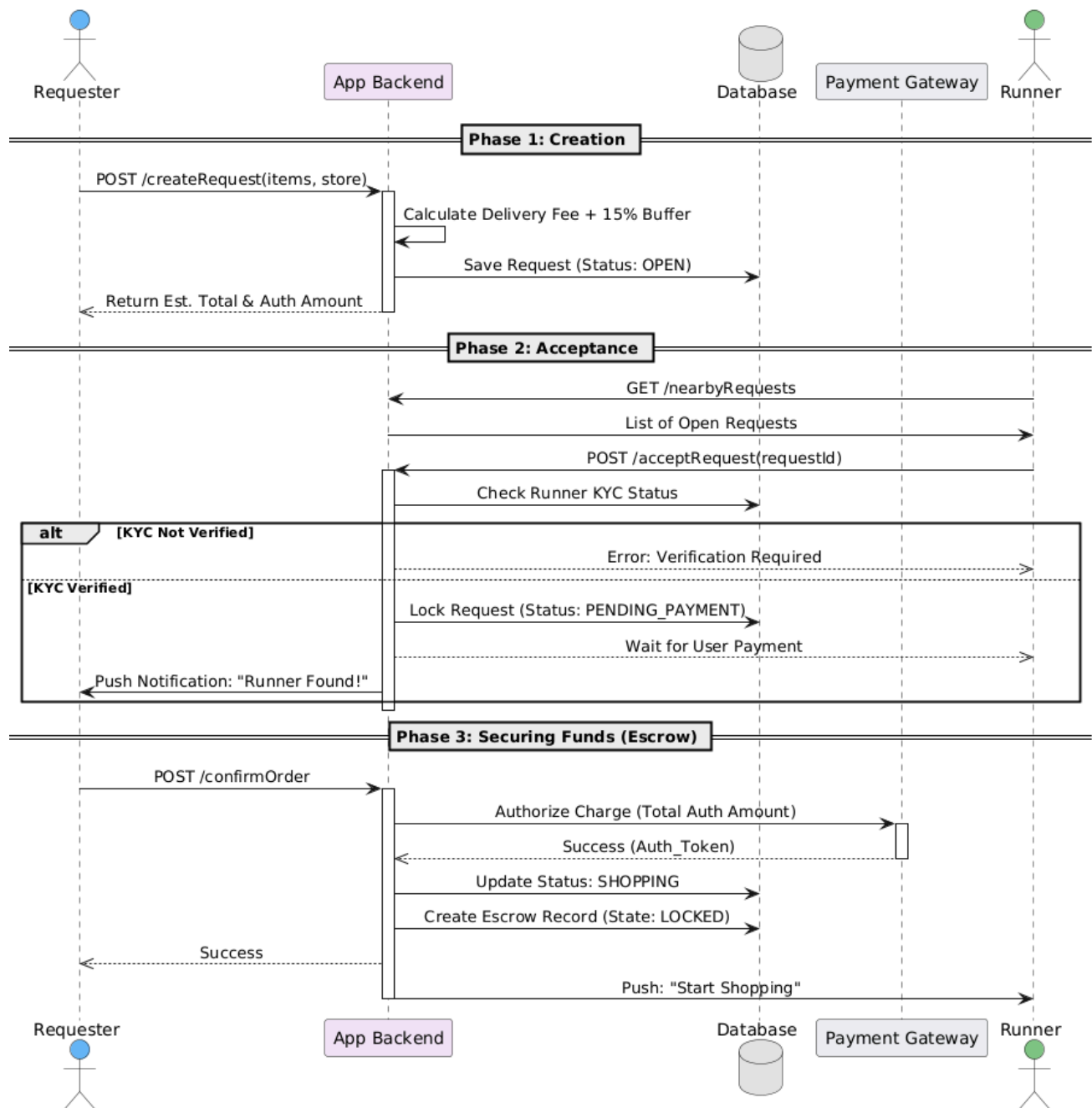
This diagram visualizes the decision loop a Runner performs when items are out of stock, detailing the interaction required for substitution approval.

Figure 3: Bill Verification & Financial Settlement Log



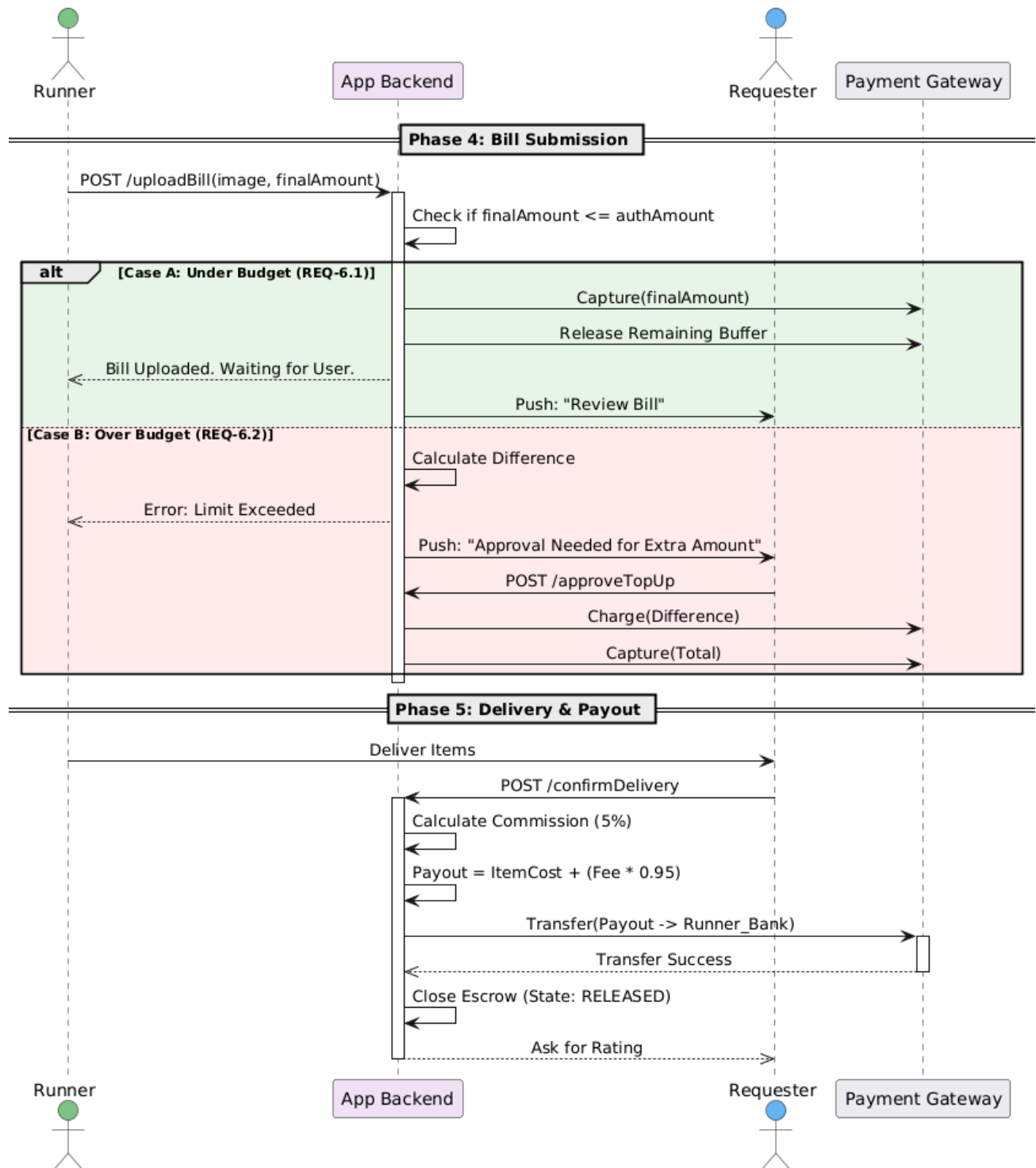
Details the critical business logic for verifying the final bill amount against the authorized budget, including the branching paths for Auto-Refunds (REQ-6.1) and Top-Up Requests (REQ-6.2).

Figure 4: Order Initiation & Escrow Locking Sequence Diagram



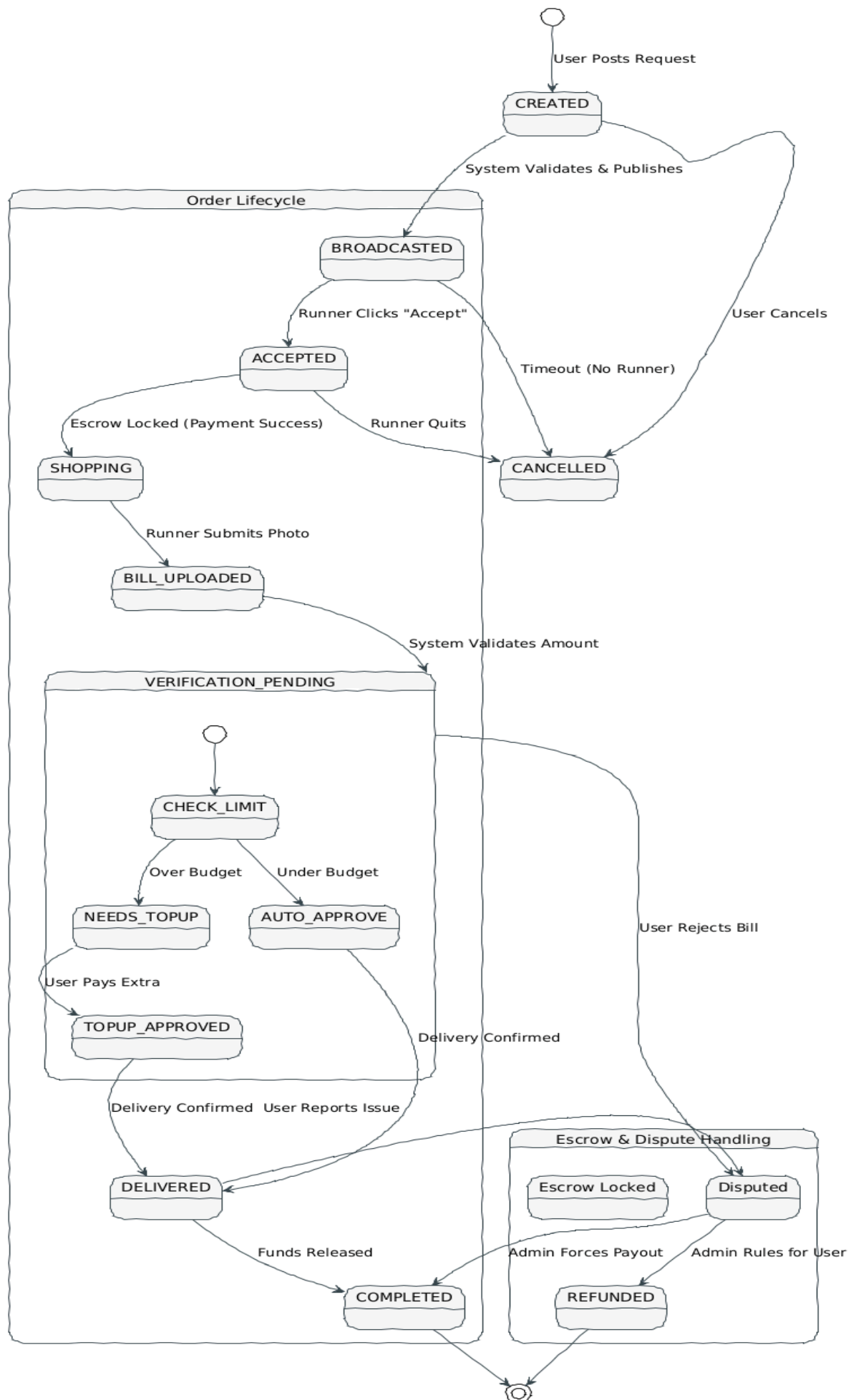
This diagram depicts the time-ordered interaction between the Requester, Backend System, and Payment Gateway to secure funds in Escrow before the Runner begins shopping.

Figure 5: Bill Finalization & Payout Release Sequence



This diagram illustrates the API calls required to capture the final payment, release the buffer amount, and transfer the commission-deducted payout to the Runner.

Figure 6: Order & Escrow Lifecycle State Transition



This diagram gives a comprehensive view of all valid states for an Order (e.g., Shopping, Delivered) and the corresponding Escrow states (e.g., Locked, Released, Frozen), including exit paths for Disputes and Cancellations.

C. Requirements Traceability Matrix

Purpose: This matrix records the evolution of requirements from the Version 1.0 to the current release Version 3.0.

Req ID (v1.0)	Requirement Description	Status in v1.0	Updates / Resolution
REQ-1	The user enters free-text items or uploads an image list.	Unchanged	Core functionality retained.
REQ-2	System suggests delivery fee; Users can adjust above the minimum floor.	Modified	Update: Added "Surge Logic." The minimum floor now increases by 10% during high-traffic hours.
REQ-3	Calculate "Total Estimated Hold Amount" (Est. Cost + Fee).	Unchanged	Formula remains the same.
REQ-4	Show requests within a configurable radius.	Modified	Update: Radius is no longer user-configurable. It is now fixed at 5km to ensure delivery speed.
REQ-5	Prevent acceptance if KYC is incomplete.	Unchanged	Strict enforcement continues for safety.
REQ-6	Deliverers can edit "Final Item Cost" after shopping.	Unchanged	Required for handling price fluctuations at the store.
REQ-7	Funds secured in escrow until delivery confirmed.	Unchanged	Critical security feature retained.
TBD-1	Exact commission percentage.	Resolved	Resolution: Commission is now fixed at 5% of the Delivery Fee.
TBD-2	Maximum allowed distance for delivery requests.	Resolved	Resolution: Maximum distance cap set to 10 km to prevent logistical issues.
N/A	New Feature: Chat System	Added (REQ-8)	New: Integrated real-time chat between Runner and Requester once a request is accepted.

Req ID(v2.0)	Requirement Description	Status in v2.0	Implementation Details
REQ-1	The user enters free-text items or uploads an image list.	Active	Core Feature (Request Creation).
REQ-2	Delivery Fee Calculation & Minimum Floor.	Active	Enforces min. fee (e.g., ₹30) + distance calculation.
REQ-3	Total Auth Amount Calculation.	Active	Formula: (Est. Cost * 1.15) + Delivery Fee.
REQ-4	Request Visibility Radius.	Active	Runner configurable (1km - 10km).
REQ-5	KYC Verification Check.	Active	Mandatory check before accepting orders.
REQ-6	Bill Finalization (Split Logic).	Active	Handles REQ-6.1 (Auto-refund) & REQ-6.2 (Top-Up).
REQ-7	Escrow Fund Security.	Active	General policy: Funds held until delivery.
REQ-8	Payment Gateway Integration	Active	Escrow: System authorizes funds via Stripe/UPI <i>before</i> shopping begins.
REQ-9	Escrow Ledger Segregation	Active	Backend: Logic to segregate funds so they cannot be withdrawn prematurely.
REQ-10	Refunds & Settlement Payouts	Active	Financial: Auto-calculates partial refunds (if bill < est) and Runner payouts.

REQ-11	In-App Masked Chat.	Active	Connects Runner/Requester without revealing phone numbers.
REQ-12	Item List Modification.	Active	Allows Runner to add/remove/substitute items strictly while the order state is SHOPPING . Locks edits upon transition to BILL_UPLOADED .
REQ-13	Substitution Blocking.	Active	Prevents bill upload if a substitution is pending approval.
REQ-14	Dispute Button.	Active	"Report Issue" freezes the Escrow Ledger immediately.
REQ-15	Dispute Evidence.	Active	Requires photo evidence/reason code to raise a dispute.
REQ-16	Admin Dashboard.	Active	Interface for Admins to force-release or refund disputed funds.